

Jess Dowd

CS 499: Computer Science Capstone

Dr. Sherri Maciosek

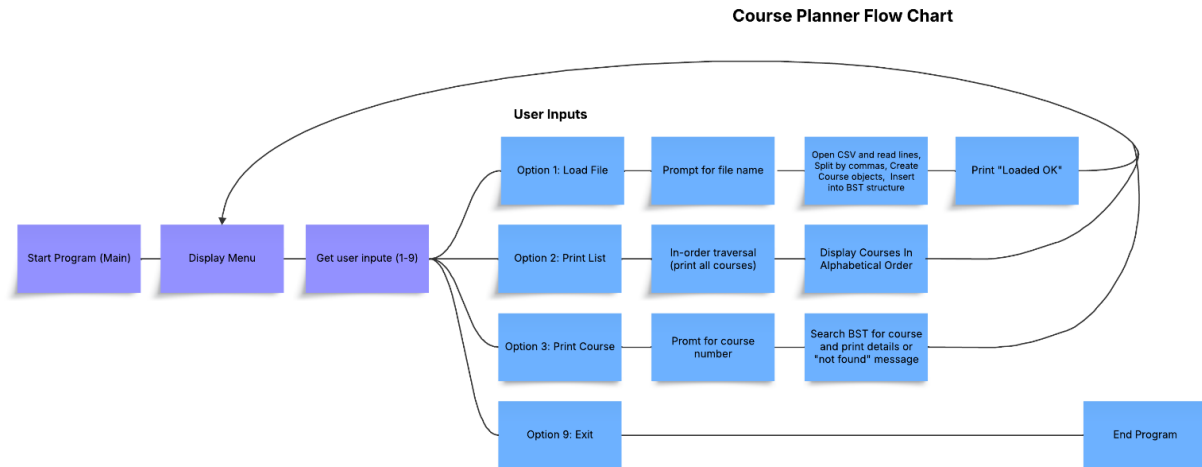
November 20, 2025

3-2 Milestone Two: Enhancement One Narrative

This artifact is a course planner written in C++ that I created in CS 300 in 2024. This app lets the user load a course data file, print the contents, search specific content, and display formatted results. For this enhancement I ported the app from C++ to python, I kept the core features and functionality but I improved readability and usability and enhanced several features and logic.

I chose to use this project for the first enhancement category of Software Engineering and Design because it is a real example of me applying my core computer science skills of using data structure, searching and sorting algorithms, and modular program design. I wanted to show my ability to translate algorithmic logic and data structures across different popular programming languages and improve on my earlier work. The result of my enhancement is a command-line course planner app built in python.

The program logic follows the flow chart and pseudocode shown below.



Pseudocode:

main:

show menu

get user input

if option 1: load file

if option 2: print all courses

if option 3: print single course

if option 9: exit

load file:

open csv file

for each line:

split by commas

create course object (number, title, prereqs)

insert into BST

print “Courses loaded successfully”

insert into BST:

if tree empty: new node = course

else compare course number and go left or right

return tree

print all courses:

do in-order traversal (left, current, right)

search course:

look through BST by course number

if found, print course title + prereqs

else print “Course not found”

I ported the logic from C++ to python and I also implemented improvements that cover important areas of the software such as structure, logic, functionality, security, commenting, and documenting.

Most of the logic is the same as the original program, but I did improve the logic and functionality around input validation and error handling by adding a boolean flag in main which checks to stop users from listing or searching courses before the data is loaded, and a

message letting them know they need to load a course, which shows good defensive programming practices.

```
Welcome to the course planner.  
1. Load course data from file.  
2. Print full course list.  
3. Print a single course and its prerequisites.  
9. Exit the program.  
What would you like to do? 4  
4 is not a valid option. Please enter a valid menu option.
```

```
Welcome to the course planner.  
1. Load Data Structure.  
2. Print Course List.  
3. Print Course.  
9. Exit.  
What would you like to do? 2  
Please load the course data first (option 1).
```

I also improved the logic and function by normalizing to upper case in the course search function by modifying the search logic in the BinarySearchTree class, which makes the program more user friendly, this allows the user to search the course in lower or uppercase without error.

```
# Enhancement: courseNumber is normalized to uppercase for case-insensitive search.  
def _search(self, node, courseNumber):  
    courseNumber = courseNumber.upper()
```

```
# Enhancement: normalize course numbers and titles.  
courseNumber = parts[0].strip().upper()  
courseTitle = parts[1].strip()
```

I improved functionality by cleaning up empty prerequisites. If the CSV has trailing commas, it could show empty strings in prerequisites so I added logic to fix that.

```
# Enhancement: clean up prerequisites by stripping whitespace
# and skipping empty entries caused by trailing commas.
raw_prereqs = parts[2:]
prerequisites = [p.strip() for p in raw_prereqs if p.strip() != ""]
```

I improved security by validating course numbers (e.g., only allow [A-Z0-9]). I improved the output wording to be clearer to the user for all options.

```
# Enhancement: validate courseNumber only contains letters, digits, or spaces.
if not all(ch.isalnum() or ch == " " for ch in courseNumber):
    print(f"Error on line {lineCount}: invalid course number '{courseNumber}'.")
    continue
```

I also had magic numbers in the menu which are not good long term because if I hard-code 1, 2, 3, 9 in ten different spots and later I add another menu option or reorder them, then I have to find and update every single one. So I improved that with an Enum which will allow changing one line for updates.

```
(class) MenuOption
if choice == MenuOption.LOAD:
```

The structure in the enhanced python version is now easier to read and modify and has a simplified structure. Before I had a struct course, a separate struct node, a BinarySearchTree class with private pointers and manual pointer handling. Now in the Python enhancement I used classes with almost no boilerplate and it handles None instead of pointers. It has less setup needed for constructors and no manual memory management. In addition the Python version uses if/elif logic and try/except validation which makes it easier to follow and modify, the older C++ version had switch statement with repeated input-clearing calls

I improved the commenting by adding the old C++ for comparison and documenting why I chose to implement it in python throughout the code, as well as documenting the changes I made and skills improved in my narrative.

I successfully met the course outcomes I planned. I met outcome two because I designed professional quality written communication that is coherent and technically sound, and appropriately adapted through my use of comments in my code that communicate the purpose and function to other developers who read the code, as well as by building in clear instructions to users for common use cases, so that users who are new to the program would know how to run it.

I met outcome 3 because I designed and evaluated computing solutions In the project using algorithms while managing tradeoffs in any design choices by re-coding the project in a different language. I showed use of important principles such as encapsulation, instantiation, classes and objects, and at each step of the code I examined how best to implement it in the new language, and why any changes might be needed to better implement it, and I explained my choices in the comments and narrative.

I met outcome 4 by porting from one language to the other which shows my understanding of OOP and data handling in multiple languages. I demonstrated my ability to use several different tools to build the code such as MSYS2, bash, VSC, Github, which is an important skill to have in many industry specific goals including my career of web development.

I improved my skills in translating logic and data structures to different programming languages. For this enhancement I focused on showing my skills in algorithmic logic and maintainability of my code across the two different languages. trade-offs in areas like performance versus maintainability among other things. I faced the usual challenges in coding a

program such as occasional minor syntax errors and debugging, and choosing which parts of the code to focus my enhancement on. I learned to improve readability when I write future code so it is easier to work with when I return to it after some time.